The Project Explained

A brief study guide to the multi-sensor fusion & trajectory planning system

What it does · how each part works · the results · the questions you will be asked

Item	Value
Reading time	≈ 45 minutes (chapters 1–3: 10 minutes for the big picture)
Companion document	Multi_Sensor_Fusion_Technical_Handover.pdf (every file, class and function)
Project location	D:\researcpaper\multi_sensor_fusion
Numbers from	configs/, outputs/ablation/results.md, outputs/evaluation/joint/report.json, model F checkpoint
Document generated	2026-10-07 13:12
Generator	docs/handover/build_brief.py

Contents

1. The Project in One Minute 3

2. Ten Ideas You Must Know 4

3. The Big Picture 5

4. Getting the Sensors to Agree 7

5. Camera → Bird's-Eye View 9

6. LiDAR and Radar Encoders 10

7. Feature Selection — the Research Idea 11

8. Fusion and 3-D Detection 13

9. Tracking and Planning 14

10. How It Was Trained 16

11. Results — and How to Talk About Them 17

12. The Live Demo 19

13. Questions You May Be Asked 20

14. Limitations and Next Steps 22

15. One-Page Cheat Sheet 23

How to use this guide

- **Chapters 1–3** give the whole picture in about ten minutes.
- **Chapters 4–10** explain each part: what it does, how, and why.
- **Chapter 11** tells you how to present the results honestly; **chapter 13** lists the questions you are most likely to get; **chapter 15** is a one-page cheat sheet of numbers.
- Every chapter ends with a **Say it like this** box: one or two sentences you can repeat in your own words.

The full technical details (every file, class and function) are in the companion *Technical Handover* PDF.

1. The Project in One Minute

What we built

A complete **perception-and-planning pipeline for a self-driving car**, trained and tested on real data from the **nuScenes** dataset. It does six things, in this order:

1. **Reads three kinds of sensors**: 6 cameras, 1 LiDAR (laser scanner) and 5 radars, recorded at the same moment on a real car in Boston and Singapore.
2. **Draws all of them on the same top-down map** around the car: the *Bird's-Eye View* (BEV), a 100 m × 100 m square cut into 200 × 200 cells of 0.5 m.
3. **Decides which LiDAR and radar information is worth keeping**: *learned feature selection*, the research part of the project.
4. **Fuses** the three sensors into one map and **detects 3-D objects** (10 classes: cars, pedestrians, trucks …), each with its position, size, heading and speed.
5. **Tracks** the objects over time (gives each one an ID).
6. **Plans** where our car should drive in the next **3 seconds** (6 points, one every 0.5 s).

A **Streamlit dashboard** replays real scenes, shows every step, and explains what the network chose.

The research question

> **Hypothesis:** *learned feature selection before LiDAR–radar fusion improves perception and planning, because it keeps the informative features and suppresses redundant or noisy ones.*

To test it, six versions of the system (Experiments **A–F**) were trained with exactly the same recipe and compared; only the sensors and the fusion design change.

The honest answer (know this by heart)

- On nuScenes-mini, feature selection did **not** beat simple concatenation (Experiment E vs C: −0.061 NDS). The hypothesis is **not supported at this data scale**.
- **Adding radar to LiDAR clearly helped** (+0.056 NDS, velocity error −0.33 m/s).
- **Adding the camera clearly helped** (+0.039 NDS); the full model F has the **best mAP** of all.
- The feature-selection gates are **interpretable**: they show which sensor features the network keeps and where. That part works and can be demonstrated live.

Three things that make this project special

What	Why it matters
All three sensors in one BEV map	Camera, LiDAR and radar become directly comparable, cell by cell.
Explainable feature selection	Every gate is a number between 0 and 1 that you can plot: "how much of this feature, here".
Runs end-to-end on a 4 GB laptop GPU (GTX 1650)	Training peaks at 3.2 GB; one frame takes about 100 ms.

> **Say it like this:** "I built a full self-driving pipeline on real nuScenes data: camera, LiDAR and radar go into one bird's-eye-view map, a small network decides which LiDAR and radar features to keep, then we fuse, detect, track and plan 3 seconds ahead. My research question was whether that learned selection helps. On the small dataset it did not beat simple concatenation, but radar and camera both helped clearly, and the selection is fully explainable."

2. Ten Ideas You Must Know

If you understand these ten words, you can follow every other chapter.

Term	Plain meaning
nuScenes	Public driving dataset by Motional. Real recordings with 6 cameras, 1 LiDAR, 5 radars, plus human-drawn 3-D boxes around every object. We use **nuScenes-mini**: 10 scenes, **323 training** and **81 validation** frames.
Key-frame / sample	A moment (2 per second) where all sensors are matched and objects are labelled. Between key-frames there are extra unlabelled *sweeps*.
Ego vehicle	Our own car. The *ego frame* is a coordinate system fixed to it: **x forward, y left, z up**, origin at the rear axle.
BEV (Bird's-Eye View)	A top-down image of the area around the car. Ours covers ±50 m in x and y, 200 × 200 cells, 0.5 m each.
Feature / channel	A number the network computes per cell. "64 channels" = 64 numbers describing each cell (like 64 colour layers instead of RGB).
Tensor shape	The size of a data block, e.g. **(B, C, H, W)** = batch, channels, height, width. (1, 64, 200, 200) is one BEV map with 64 features per cell.
LiDAR	Spins lasers and measures distance: tens of thousands of precise 3-D points per sweep. Excellent geometry, no speed, sparse far away.
Radar	Radio waves. Very few points (≈ 400–800 per frame) and imprecise in angle, **but it measures speed directly** (Doppler) and works in rain and fog.
Fusion	Combining the sensors into one representation.
Ablation study	Remove or swap one part at a time and measure the change. Experiments A–F are our ablation.

The metrics (you will be asked):

Metric	Meaning	Better
mAP	Mean Average Precision: are the detected boxes in the right place and class? A box counts as correct if its centre is within 0.5 / 1 / 2 / 4 m of a true box.	higher
NDS	nuScenes Detection Score: half mAP, half box quality (position, size, heading, velocity, attribute errors). The headline number.	higher
mAVE	Mean velocity error of the detected objects, in m/s.	lower
ADE / FDE	Planning: average / final distance between our planned path and the path the human driver actually drove (metres).	lower
Collision rate	Share of plans where our car's footprint would overlap another object's *real* future position.	lower

$$NDS = \frac{1}{10}\left[5\,\mathrm{mAP} + \sum_{k=1}^{5}(1 - \min(1,\ \mathrm{err}_k))\right]$$

Say it like this: "Everything is converted into a top-down grid around the car, 200 by 200 cells of half a metre. Each sensor gives 64 numbers per cell. Detection is judged with the official nuScenes score, NDS, and planning by how far our path is from the human driver's path and whether it would collide."

3. The Big Picture

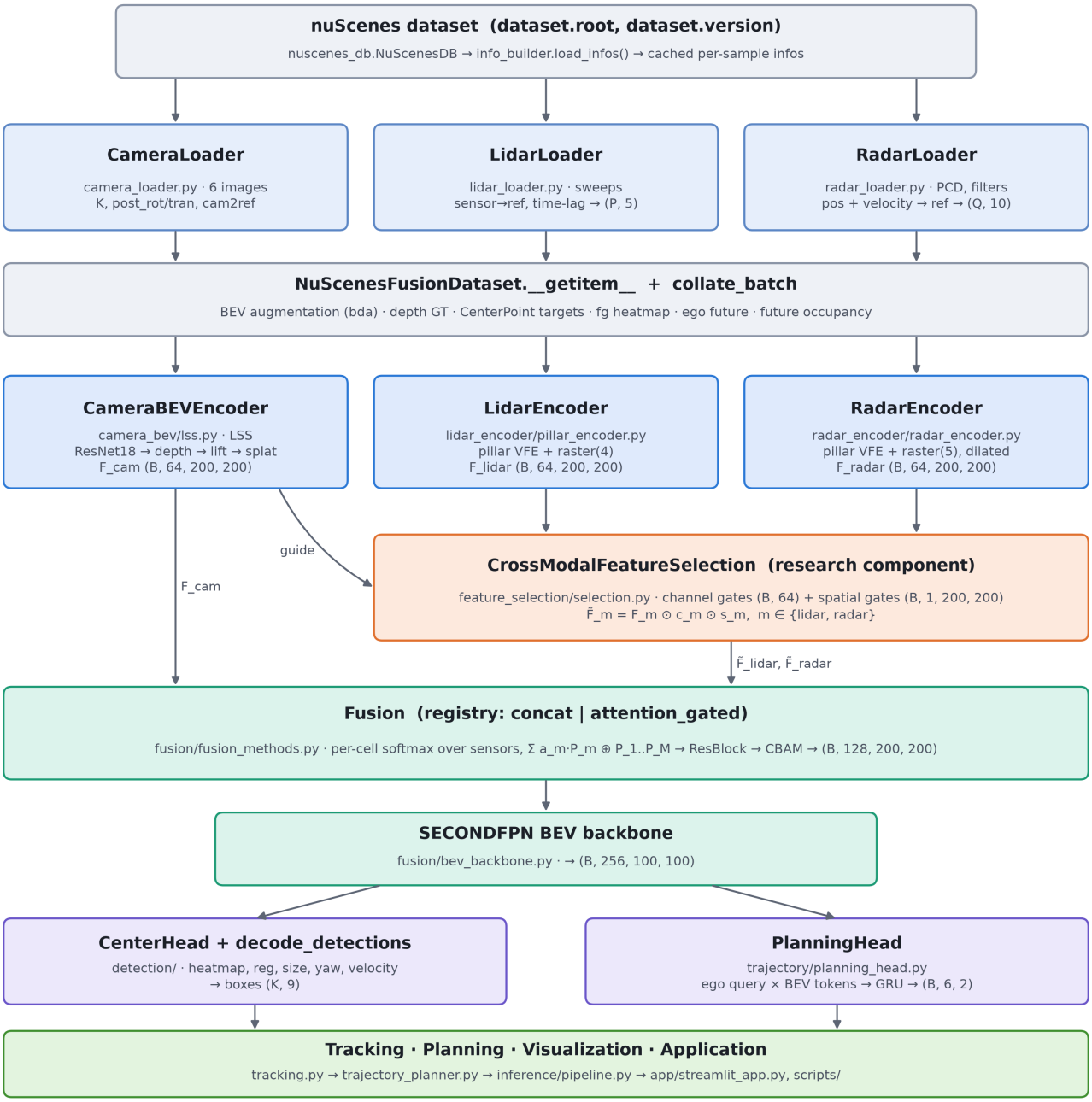


Figure 1 – System architecture of the proposed model (Experiment F).

The pipeline in nine steps

#	Step	What happens	Result (one frame)
1	Load & synchronise	Read all sensors of one key-frame; move everything into the ego frame at the LiDAR time.	images, point clouds, labels
2	Camera → BEV	Guess a depth for every image feature and "drop" it into the top-down grid (LSS).	(64, 200, 200)
3	LiDAR → BEV	Group points into vertical columns ("pillars"), learn 64 features per cell.	(64, 200, 200)
4	Radar → BEV	Same for radar, keeping its velocity; spread the sparse points with dilated convolutions.	(64, 200, 200)
5	**Feature selection**	Gates decide *which* LiDAR/radar features to keep and *where*. Camera acts as guide.	gated LiDAR + radar

#	Step	What happens	Result (one frame)
6	Fusion + backbone	Weigh the sensors per cell and mix them; a CNN refines the map.	(128, 200, 200) → (256, 100, 100)
7	Detection	Find object centres as peaks in a heatmap; read box size, heading, speed at each peak.	boxes (K, 9) + scores + classes
8	Tracking	Match boxes to those of the previous frame to keep IDs.	tracks with IDs
9	Planning	Learned head (or classical A* + iLQR) outputs our future path.	6 waypoints (x, y), 3 s

A box has **9 numbers**: centre (x, y, z), size (length, width, height), heading (yaw), velocity (vx, vy).

What one frame looks like inside the network

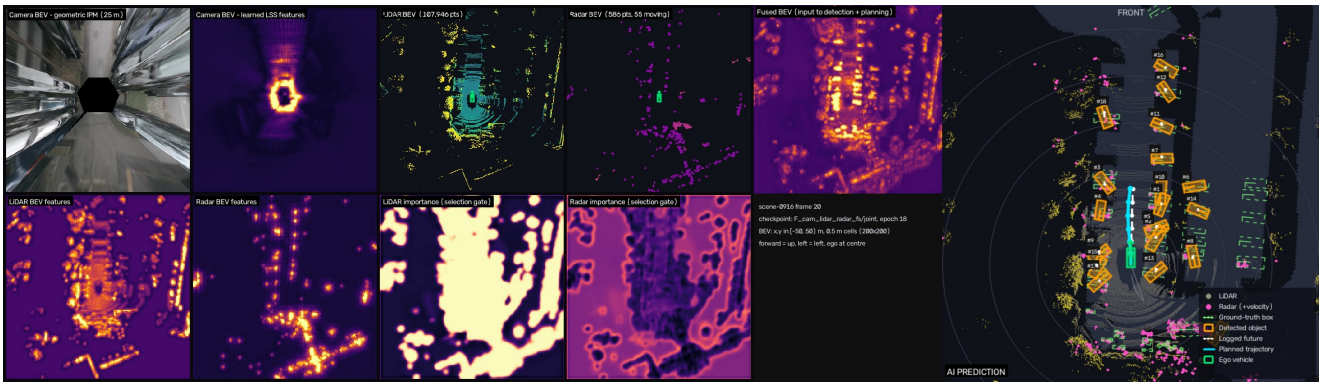


Figure 2 – Real example (scene-0916, validation): every BEV layer of the pipeline. Top: camera IPM, learned camera BEV, raw LiDAR, raw radar, fused BEV. Bottom: LiDAR and radar features, their selection gates (bright = kept), and the final detections with the planned path.

How to read Figure 2: the **LiDAR gate** (bottom, third) is bright almost everywhere there are LiDAR returns: the network keeps LiDAR where LiDAR sees something. The **radar gate** (bottom, fourth) is smoother and dimmer: radar is kept as broad context, not point by point.

> **Say it like this:** "Each sensor is turned into the same 200 by 200 top-down map. Before fusing, a selection network gates LiDAR and radar. Then the maps are fused, a CenterPoint-style head finds the objects, a tracker gives them IDs, and the planner outputs six waypoints for the next three seconds."

4. Getting the Sensors to Agree

Problem. Every sensor has its own position on the car and fires at a slightly different moment (cameras up to ≈ 45 ms, radars up to ≈ 37 ms away from the LiDAR). At 15 m/s the car moves about 0.7 m in 45 ms, enough to misplace objects.

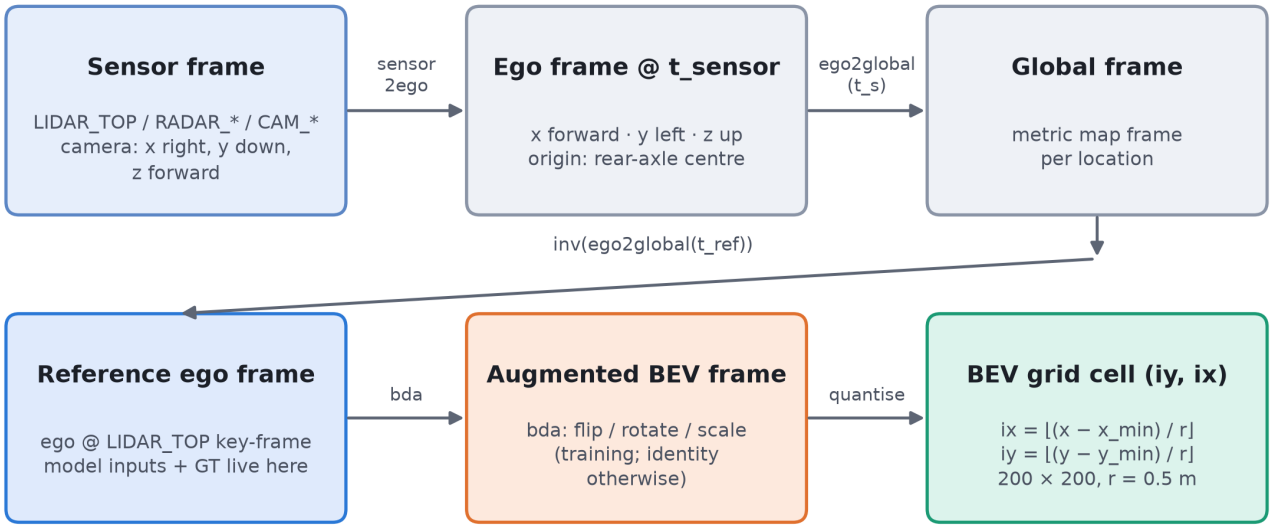
Solution. Every point is moved through a chain of 4 × 4 transformation matrices (rotation + translation):

$$T_{sensor \to ref} = \left(T^{(t_{ref})}_{ego \to global}\right)^{-1} T^{(t_s)}_{ego \to global} \, T_{sensor \to ego}$$

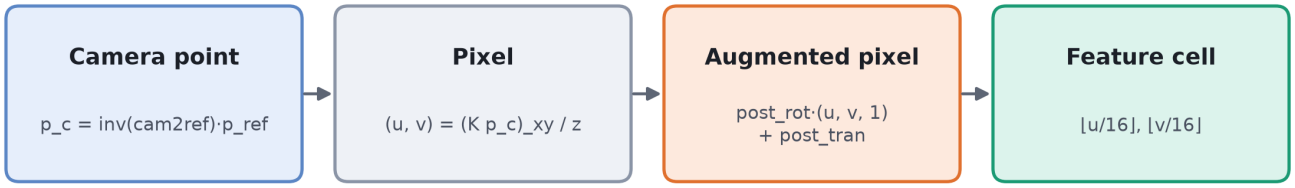
Read it right to left: sensor → car (at the sensor's own time) → world (fixed) → car (at the LiDAR key-frame time). Going through the fixed **global/world frame** is what cancels the car's motion. It is computed once per sweep in `datasets/info_builder.py`.

```
sensor2ref = inv(ref_ego2global) · ego2global(t_sensor) · sensor2ego
```

computed per sweep in datasets/info_builder.py: _sensor_record



Camera projection (camera_view.project_points, camera_loader.lidar_depth_maps):



LSS lift (lss.get_geometry) runs it backwards:

```
(u', v', d) = post_rot⁻¹((u, v, d) − post_tran)

p_ref = R_cam2ref · K⁻¹ · [u'd, v'd, d]ᵀ + t_cam2ref ;  p_bev = bda · p_ref
```

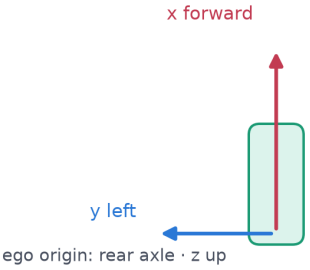


Figure 3 – The coordinate frames and the order in which they are applied.

Three details worth knowing

- **BEV cell of a point**: $i_x = \lfloor (x - x_{min}) / 0.5 \rfloor$, and the same for y. Rows are y, columns are x.

- **Radar velocity** is a direction, not a position, so only the **rotation** is applied (no translation).
- **Ground-truth boxes** are given in world coordinates; they are moved into the same reference frame. Their heading is re-computed from the rotated direction.

Say it like this: "Sensors fire at different times while the car moves, so every measurement goes sensor → car → world → car-at-reference-time. The world frame doesn't move, so passing through it removes the ego-motion. Velocities are only rotated, because they are directions."

5. Camera → Bird's-Eye View

The difficulty. A camera image is flat. It shows *what* is there (a car, a person) but not *how far away* it is. To place camera information on a top-down map we must estimate depth.

The method: Lift–Splat–Shoot (LSS) with BEVDepth-style depth supervision (`models/camera_bev/lss.py`)

1. **Backbone.** A ResNet-18 (pre-trained on ImageNet) turns each of the 6 images (128 × 352 px on our GPU) into a feature map of 8 × 22 cells (each cell = 16 × 16 pixels).
2. **Depth net.** For every feature cell it predicts a **probability over 59 depth bins** (1 m to 60 m) and 64 context features. It is *camera-aware*: it also receives the camera's calibration, so it knows which lens it is looking through.
3. **Lift.** Each context feature is copied along its viewing ray, multiplied by the depth probability: much at the likely depth, little elsewhere.
4. **Splat.** All those 3-D points are dropped into their BEV cells and summed.
5. **Refine.** Two small residual blocks clean the map → **(64, 200, 200)**.

Depth supervision. During training, LiDAR points are projected into the images; the nearest LiDAR depth in each feature cell becomes the "correct" depth bin, and the depth net is trained on it (binary cross-entropy). This is what makes the depth guesses meaningful.

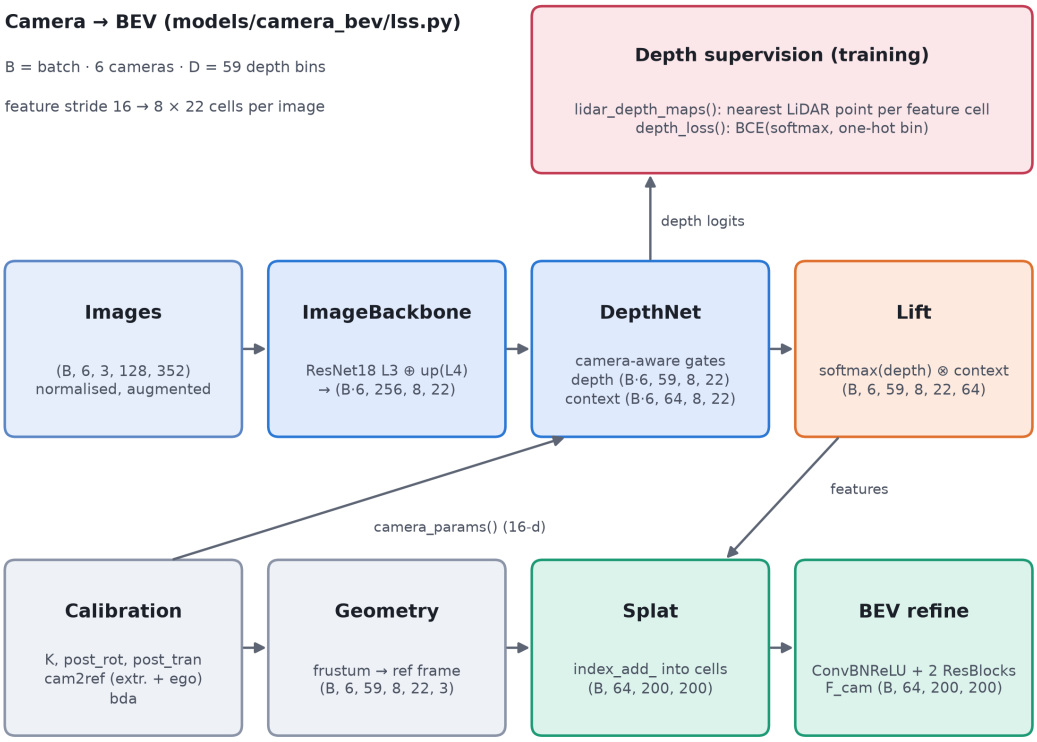


Figure 4 – Camera → BEV pipeline with the real tensor sizes.

Analogy. Imagine pointing a torch from the camera through each pixel and spraying paint along the beam: more paint where the network believes the object is. Seen from above, the paint forms the camera's top-down map.

Why the camera BEV is brightest near the car: more rays pass through nearby cells, and the sum adds them up. It is a property of LSS, not a bug.

> **Say it like this:** "Cameras have no depth, so for every image feature the network predicts a distribution over 59 depth bins, spreads the feature along the camera ray according to it, and sums everything into the top-down grid. LiDAR gives the true depth during training to teach it."

6. LiDAR and Radar Encoders

LiDAR → BEV (`models/lidar_encoder`)

Input. About 5 LiDAR sweeps (on the GTX 1650 profile) merged into one cloud. Each point has **5 values: x, y, z, intensity, time-lag** (how old the sweep is). Points closer than 1.5 m (our own car) are removed.

Method: dynamic PointPillars.

1. Every point is assigned to the BEV cell (*pillar* = vertical column) it falls into.
2. A small shared network (PointNet: MLP, then max over the pillar's points) turns all points of a pillar into one feature vector. "Dynamic" = no limit on points per pillar, nothing is thrown away.
3. Four easy-to-read **raster channels** are added: point density, max height, mean height, intensity.
4. Two residual blocks → **(64, 200, 200)**.

Radar → BEV (`models/radar_encoder`)

Input. 5 radars, about 5 sweeps, typically only **400–800 points per frame**. Each point has **10 values**: x, y, z, RCS (how strongly it reflects), compensated velocity (vx, vy), raw velocity, dynamic-property flag and time-lag. The nuScenes default filters remove invalid and ambiguous points.

Method.

1. Same pillar idea as LiDAR.
2. **Dilated convolutions** (dilations 1, 2, 4) spread each sparse point's information to its neighbourhood (receptive field ≈ 31 cells ≈ 15 m), because radar points are few and inaccurate in angle.
3. Five raster channels: density, **vx, vy**, RCS, time-lag. The velocity is kept explicitly.
4. Output **(64, 200, 200)**.

Why use both?

	LiDAR	Radar	Camera
Strength	precise 3-D shape and position	**measures speed**, long range, weather-robust	colour, texture, **what** the object is
Weakness	no speed, sparse far away, costly	very sparse, poor angle, "ghost" reflections	no depth, sensitive to light
In our results	best single sensor (A: NDS 0.060)	weak alone (B: 0.053, mAP 0.007)	adds +0.039 NDS on top of L+R (F vs E)

Say it like this: "LiDAR points are grouped into vertical pillars and a small PointNet learns 64 features per cell. Radar uses the same idea but keeps the Doppler velocity as explicit channels and uses dilated convolutions because it has only a few hundred points. LiDAR gives shape, radar gives speed, the camera gives meaning."

7. Feature Selection — the Research Idea

The problem it solves

Simple fusion (concatenation) hands **every** LiDAR and radar feature at **every** cell to the next layer, including redundant ones (both sensors saying "something is here") and noisy ones (radar ghosts, LiDAR returns from trees). The idea: **let a small network decide what to keep before fusing.**

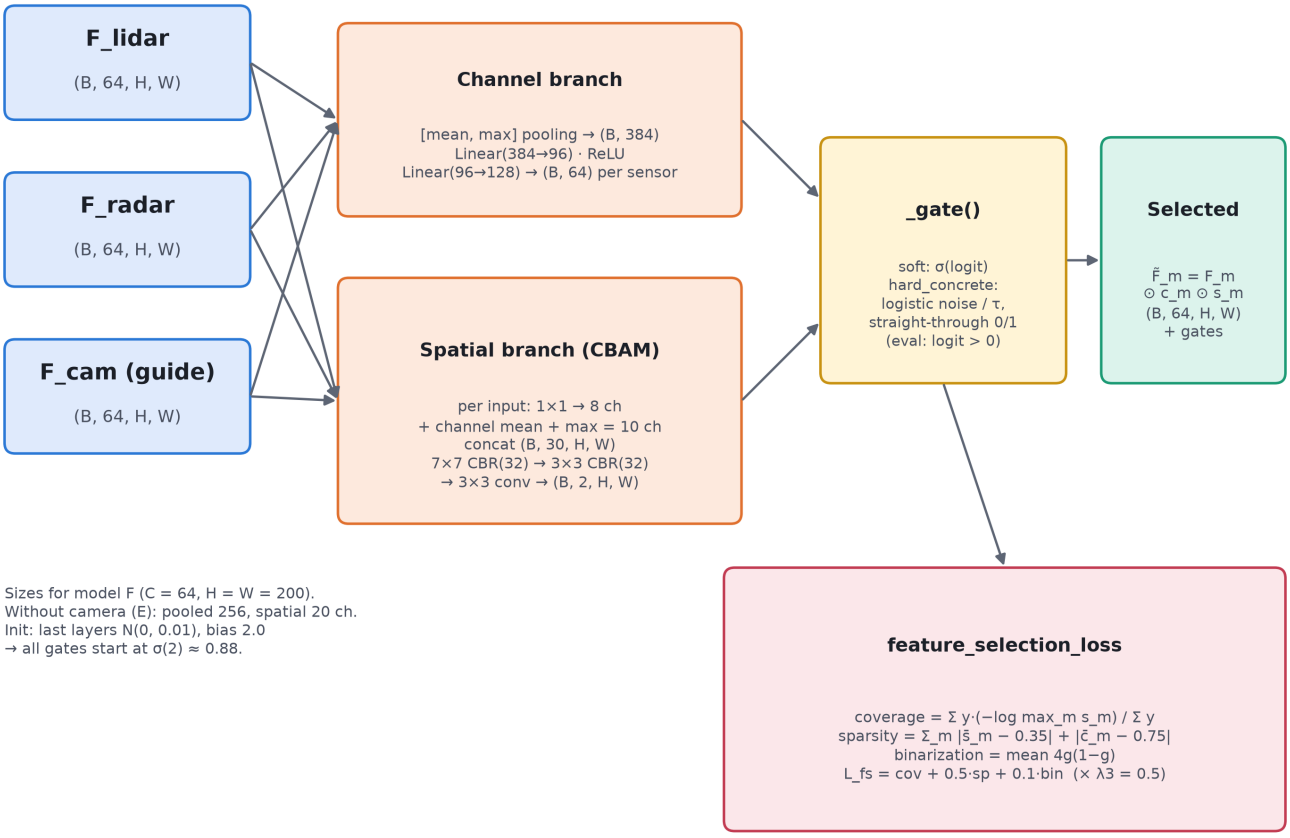


Figure 5 – The feature-selection module with its tensor sizes and losses.

How it works (`models/feature_selection/selection.py`, "CG-CMFS")

For each of **LiDAR and radar** the module predicts two *gates*: numbers between 0 (drop) and 1 (keep).

Gate	Size	Question it answers
Channel gate c	64 numbers per sensor	*Which* features of this sensor are useful in this scene?
Spatial gate s	200 × 200 map per sensor	*Where* is this sensor informative?

$$\tilde{F}_m = F_m \odot c_m \odot s_m \qquad m \in \{\text{lidar},\ \text{radar}\}$$

(⊙ = element-wise multiplication: every feature is scaled by its channel gate and its cell's spatial gate.)

- **Camera-guided:** the camera BEV is an *input* to both gate networks (it provides "what is where"), but the camera itself is not gated: it is the guide.
- **Channel branch:** average- and max-pool each of the 3 maps to 2 × 64 numbers (384 in total), then a small 2-layer network outputs the 64 + 64 channel gates (like *Squeeze-and-Excitation*).

- **Spatial branch:** compress each map to 10 channels, stack them (30), then 7 × 7 and 3 × 3 convolutions output the 2 spatial gates (like *CBAM* spatial attention).
- **Gates start open:** the last layer's bias is 2, so every gate starts at σ(2) ≈ 0.88. Training begins as "keep everything" and learns what to close.
- **Tiny:** the whole module has **0.11 M parameters** (0.5 % of the 22.6 M model) and costs ≈ 7 ms.

How it is trained: three extra losses

$$L_{fs} = L_{coverage} + 0.5\,L_{sparsity} + 0.1\,L_{binarization}$$

Loss	What it does	Why
Coverage	At true object centres, at least one sensor's spatial gate must be open.	Never throw away the objects.
Sparsity	Pushes the average gate towards a target: keep ≈ **35 % of cells** and **75 % of channels**.	Forces real *selection*, not "keep all".
Binarization	Penalises gates near 0.5 (term 4·g·(1−g)).	Gates become clear yes/no → easy to interpret.

An optional **hard-concrete** mode makes the gates truly 0 or 1 during training (with random noise and a "straight-through" gradient trick). The default is **soft** gates.

Why not a Transformer?

Cross-attention over a 200 × 200 grid means 40 000 tokens → 1.6 billion attention entries. That does not fit a 4 GB GPU. Channel + spatial gating is cheap and every gate value can be plotted directly.

What the network actually learned (model F, final weights)

- About **34 % of LiDAR** features and **11 % of radar** features pass the gates.
- LiDAR spatial gate ≈ **0.99 on cells with LiDAR returns**, ≈ 0.42 elsewhere.
- **Radar velocity (vx, vy), RCS, density and time-lag channels are all kept** (gates 0.96–0.99).
- With the camera present, LiDAR **mean height** is switched off (gate 0.001) while **max height** is kept (0.98). Without the camera (model E) mean height stays on (0.89).
- The radar spatial gate is *lower on the exact radar points* (0.14–0.18) than around them (0.35–0.38): the network uses radar as broad context. A plausible explanation is radar's poor angular accuracy, but this was not tested.

Say it like this: "For LiDAR and radar the network learns two gates between 0 and 1: a channel gate (which of the 64 features) and a spatial gate (which cells). The camera BEV guides both. Extra losses make sure objects are always covered, that only about a third of the cells are kept, and that gates become clearly on or off. It costs 0.5 % of the parameters and every gate can be shown."

8. Fusion and 3-D Detection

Two ways to fuse (`models/fusion/fusion_methods.py`)

Method	How	Used in
Concatenation	Stack all maps on top of each other, one 3 × 3 convolution + residual block.	A, B, C
Attention-gated	For every cell, predict a softmax weight per sensor ("trust LiDAR 70 %, radar 20 %, camera 10 % *here*"), take the weighted sum, mix it with the individual maps, refine with CBAM.	D, E, F

Difference between attention fusion and feature selection (a common question): attention fusion *shares* trust between sensors per cell (the weights add up to 1). Feature selection acts *before* that, per sensor, on channels **and** cells, and can turn a feature off in both sensors at once.

Backbone (SECOND-FPN). A standard 2-D CNN refines the fused map: (128, 200, 200) → **(256, 100, 100)**, one cell = 1 m.

Detection head (CenterPoint style, `models/detection/center_head.py`)

1. **Heatmap**: for each of the 10 classes, a 100 × 100 map whose peaks are object centres. Trained with a Gaussian "blob" around every true centre (Gaussian focal loss).
2. **Regression at each peak**: sub-cell offset, height z, size (log l, w, h), heading as (sin, cos), velocity (vx, vy).
3. **Decoding**: take the strongest peaks with score ≥ 0.1, convert them to boxes, remove duplicates with circle-NMS (radius per class, e.g. car 1 m, pedestrian 0.25 m).

Deep supervision: each sensor also gets a small auxiliary heatmap head during training (L_fusion), so every encoder learns to see objects on its own.

> **Say it like this:** "The fused map goes through a SECOND-FPN backbone to a 100 by 100 map. A CenterPoint head predicts one heatmap per class; every peak is an object, and at that peak we read its offset, height, size, heading and velocity."

9. Tracking and Planning

Tracking (`planning/tracking.py`)

Simple and standard (CenterPoint-style greedy tracking):

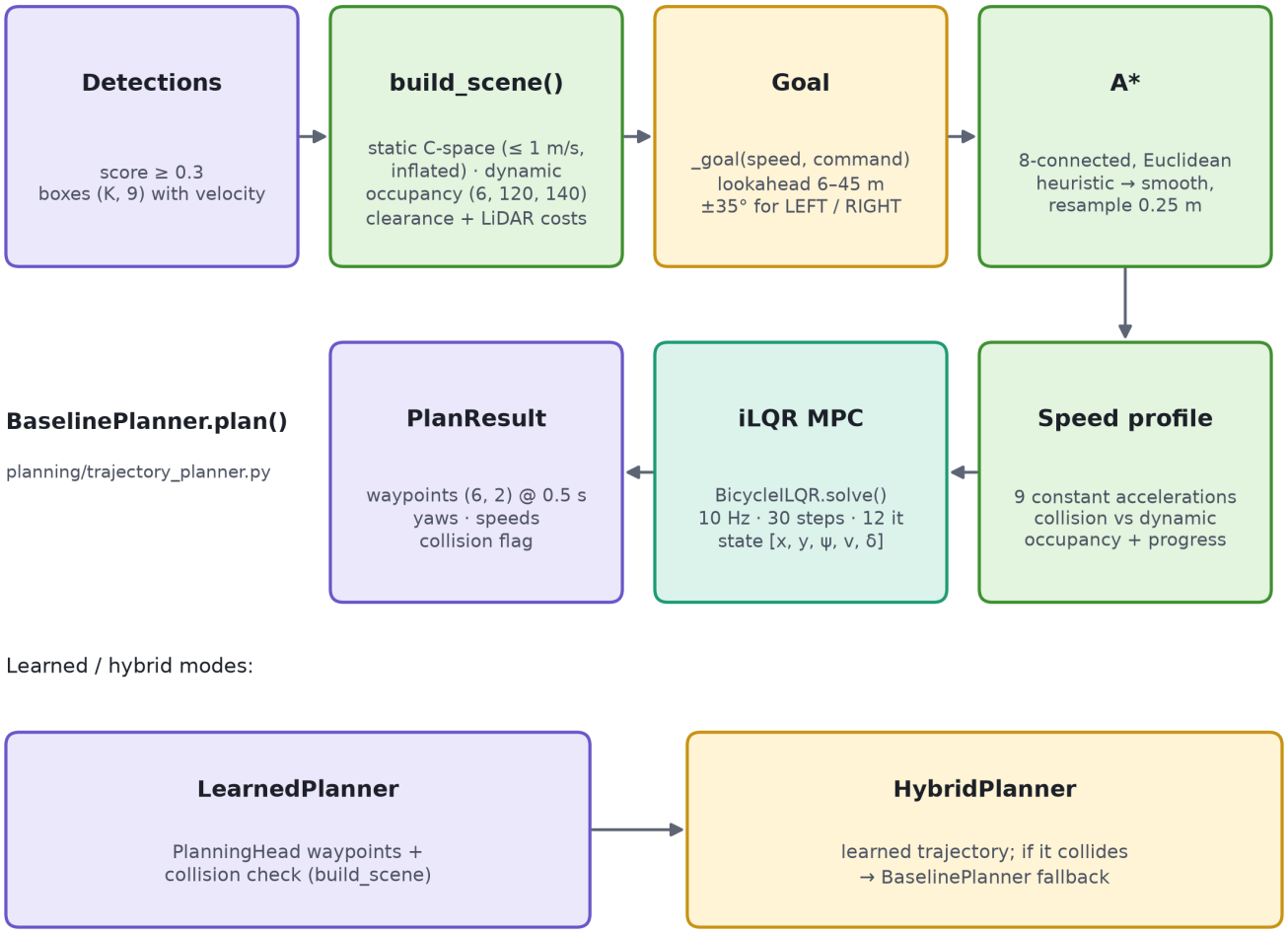
1. Move this frame's detections to world coordinates.
2. Predict where each existing track is now, using its velocity.
3. Match the highest-scoring detection first to the nearest track of the same class within a distance limit (car 4 m, pedestrian 1 m …). Unmatched detections start new tracks.

Result: persistent **IDs**, smoothed velocities and short motion histories (shown in the dashboard).

Three planners (`planning/trajectory_planner.py`)

All output **6 waypoints, one every 0.5 s = 3 s**, in the ego frame.

1. Classical planner (BaselinePlanner): no learning, fully explainable.



Learned / hybrid modes:

Figure 6 – The classical planner.

- Builds an **occupancy grid** in front of the car: static obstacles now, moving objects predicted for the next 6 steps with their velocity, plus a soft cost on raw LiDAR obstacle points (catches missed detections).
- Picks a **goal** from the current speed and the navigation command (left / straight / right).
- **A*** finds a collision-free path; a **speed profile** is chosen from 9 constant accelerations; an **iLQR MPC** controller on a kinematic bicycle model (steering + acceleration) smooths it into a drivable trajectory.

2. Learned planner (PlanningHead): a small network on the BEV map.

- An "ego query" (speed, yaw rate, acceleration, command) **looks at** the BEV map with cross-attention.
- A GRU outputs a **correction** for each of the 6 steps, added to a **constant-velocity guess**. Learning only the correction is what makes it trainable with 323 training frames.

3. Hybrid: use the learned path; if the classical collision check finds a collision, fall back to the classical planner.

The navigation command (left / straight / right) comes from where the human actually went (lateral offset of at least 2 m at the end of the logged future). This is the standard *open-loop* nuScenes protocol, like a sat-nav instruction; it does not reveal the path itself.

> **Say it like this:** "A greedy tracker keeps object IDs. For planning I have a classical planner (occupancy grid, A*, speed profile, iLQR on a bicycle model) and a learned planner that attends over the BEV map and predicts corrections to a constant-velocity guess. Both output six waypoints for three seconds."

10. How It Was Trained

One total loss

$$L = 1.0\,L_{det} + 1.0\,L_{traj} + 0.5\,L_{fs} + 0.25\,L_{fusion} + 1.0\,L_{depth}$$

Part	Meaning
L_det	Heatmap (focal) + box offset, size, heading, velocity errors
L_traj	Distance to the human path + collision with real future objects + smoothness + reaching the goal
L_fs	Feature-selection losses (coverage, sparsity, binarization)
L_fusion	Auxiliary per-sensor heatmaps
L_depth	Camera depth vs LiDAR depth

Two training recipes

- **7-stage curriculum** (for the best single model): camera → LiDAR → radar (each alone) → fusion with frozen encoders → detection fine-tuning → planner → end-to-end. Each stage starts from the previous one's weights.
- **One-stage "joint" training** (used for the **ablation A–F**): all losses together, from scratch, 20 epochs, seed 42, AdamW, learning rate $2 \cdot 10^{-4}$ with cosine decay. Identical for every experiment, so the comparison is fair.

Making it fit a 4 GB GPU (GTX 1650)

Trick	Effect
Batch 1 × gradient accumulation 4	Behaves like batch 4 with the memory of batch 1.
Images 128 × 352 (instead of 256 × 704), 5 sweeps	4× fewer camera pixels, fewer points.
Gradient checkpointing	Recomputes activations instead of storing them.
fp32 instead of fp16	On this GPU + cuDNN 9.10, fp16 3 × 3 convolutions with 256 input channels return **NaN**, and fp16 was 3.4× *slower* (no tensor cores). A self-test detects this automatically.

Result: training peak **3.2 GB**; inference **0.77 GB, ≈ 100 ms per frame**.

Which weights are evaluated? The **last epoch** (`last.pt`) of every experiment. Picking the "best" epoch on the validation set would be tuning on the test data and would favour some experiments by luck.

> **Say it like this:** "One weighted loss trains detection, planning, selection, auxiliary heads and camera depth together. For the fair ablation every experiment used the same 20-epoch joint recipe and I evaluate the final epoch. To fit 4 GB I used batch 1 with gradient accumulation, smaller images, gradient checkpointing and fp32, because fp16 produced NaNs on this GPU."

11. Results — and How to Talk About Them

Setting: nuScenes-mini, 81 validation frames (2 scenes), 20 epochs, one seed, GTX 1650, final-epoch weights.

Exp	Sensors / design	NDS ↑	mAP ↑	mAVE ↓	ADE ↓	Collision ↓	ms
A	LiDAR only	0.060	0.032	1.10	1.37	8.7 %	45
B	Radar only	0.053	0.007	1.48	1.26	11.6 %	43
C	LiDAR + Radar, concat	**0.116**	0.035	**0.77**	1.36	7.2 %	63
D	LiDAR + Radar, attention	0.070	0.021	1.42	1.28	7.2 %	67
E	LiDAR + Radar, feature selection	0.055	0.027	1.03	1.51	7.2 %	71
F	Camera + LiDAR + Radar, feature selection	0.094	**0.044**	0.91	1.53	13.0 %	103
–	constant-velocity planner	–	–	–	1.77	14.5 %	–

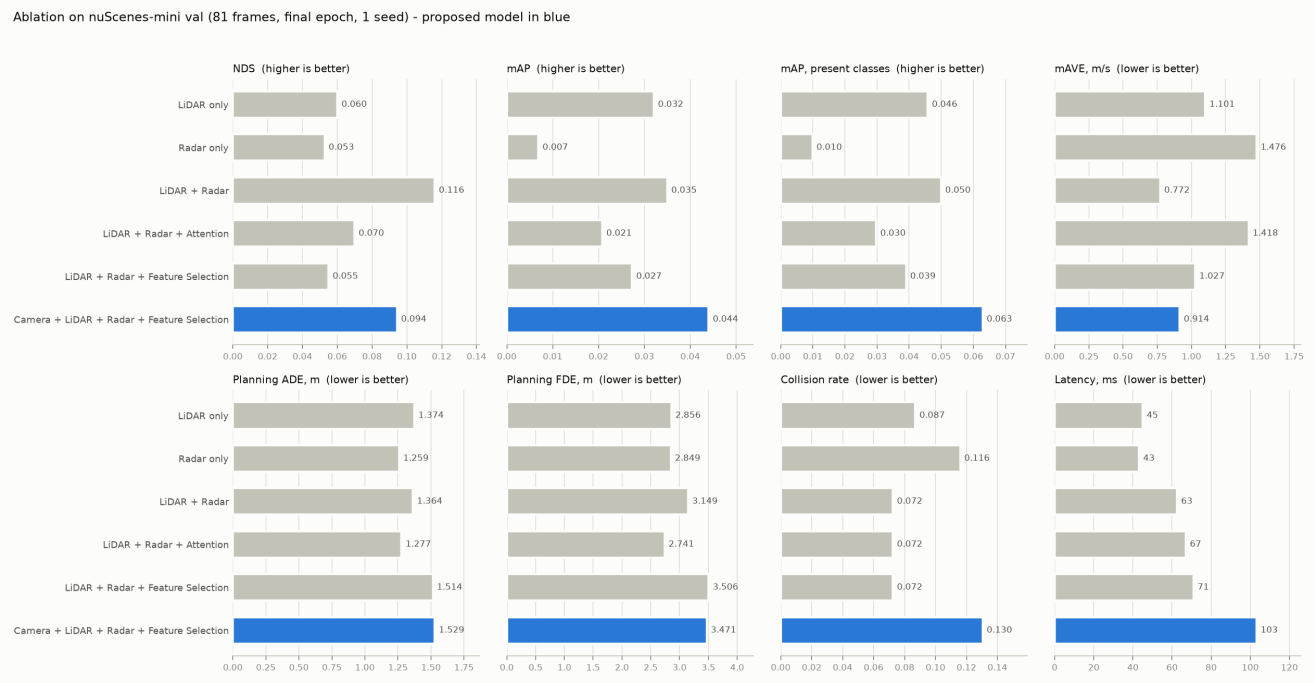


Figure 7 – Ablation chart (outputs/ablation/ablation_metrics.png).

The five findings

1. **Feature selection did not beat concatenation** here: E − C = −0.061 NDS. Against attention-only fusion it is mixed (E − D: −0.015 NDS but +0.007 mAP and 0.39 m/s *lower* velocity error).
2. **Radar helps LiDAR** (C − A): +0.056 NDS, −0.33 m/s velocity error. The most consistent effect.
3. **The camera helps** (F − E): +0.039 NDS, +0.017 mAP. F has the best mAP.
4. **Every learned planner beats constant velocity** (ADE 1.26–1.53 m vs 1.77 m).
5. **The gates are interpretable** and show sensible behaviour (previous chapters).

Official check. Model F was also scored with the **official nuScenes devkit**: NDS **0.1028**, mAP **0.0459** (our own implementation: 0.0943 / 0.0440; the small gap comes from the export score threshold, 0.05 vs 0.1). Per class (our implementation): car AP 0.33, pedestrian 0.09, truck 0.02; bus, motorcycle, bicycle and traffic cone 0. Three classes (construction vehicle, trailer, barrier) do not occur in the 2 validation scenes, which is why "mAP over present classes" (0.063) is also reported.

Why the absolute numbers are low (and why that is fine)

- **323 training frames.** Full nuScenes has 28 130, about 87 times more. Published systems trained on full nuScenes with big GPUs report NDS around 0.6–0.7.
- Small images (128 × 352), few sweeps, 20 epochs, a 4 GB GPU.
- 81 validation frames from 2 scenes, one seed: small differences can be noise.

The goal on mini is a **working, honest pipeline** and a **fair relative comparison**, not a leaderboard score.

Possible reasons feature selection did not help (not yet tested)

- With so little data, the sparsity target (keep 35 % of cells) may remove information the model still needs. E's validation NDS peaked at epoch 6 and then declined, a sign of over-regularisation.
- Noise from the tiny validation set.

Say it like this: "On mini, feature selection did not beat simple concatenation, so my hypothesis is not supported at this scale. But the study shows clearly that radar helps LiDAR, especially for velocity, and that the camera adds the best mAP. The numbers are low because mini has 323 training frames; the official devkit confirms the evaluation. The next step is full nuScenes with several seeds."

12. The Live Demo

Start it (from `D:\researcpaper\multi_sensor_fusion`, with the project's Python environment):

```
python app/streamlit_app.py
```

The browser opens at http://localhost:8501. Close any training run first; the dashboard needs the GPU.

Mode	Shows	What to say
1 · Dataset Viewer	Raw cameras, LiDAR, radar velocity arrows, true boxes.	"Real, synchronised nuScenes data in one vehicle frame."
2 · Ground Truth Simulation	Human labels + classical planner on perfect perception.	"Upper bound: what planning can do if perception were perfect."
3 · AI Autonomous Driving	The trained network on every frame (banner **MODEL PREDICTION**).	Walk through the stage strip: camera → BEV, LiDAR + radar, selection, fusion, detection, planning.

Best moments to show

- Use the **validation scenes scene-0103 and scene-0916**: the model never saw them during training.
- Pause on a busy frame → **Explain Feature Selection**: raw → importance → selected for each sensor, the sensor-dominance map, channel-gate bars (e.g. radar velocity kept), per-class trust table. "Every number here comes from the network's own gates for this frame."
- **Compare fusion methods**: runs all six trained models on the same scene.
- **Ablation study** panel: the A–F table and the hypothesis tests.

Built-in honesty: without a trained checkpoint, mode 3 says **GROUND TRUTH / DATASET** and explains why; an untrained network shows a red **UNTRAINED** banner; training scenes are flagged.

Other useful commands:

```
python main.py bev --scene scene-0916 --frame 20

python main.py infer --scene scene-0103 --video
```

(BEV gallery images in `outputs/bev/`; a replay video in `outputs/inference/`, a good backup if the laptop misbehaves.)

13. Questions You May Be Asked

About the idea

Q: Why convert everything to a bird's-eye view? A: It is a common space for all sensors, objects keep their real size at any distance (unlike in an image), and planning happens on the ground plane anyway. Fusion becomes a simple per-cell operation.

Q: Why three sensors? Isn't LiDAR enough? A: LiDAR gives shape but not speed; radar measures speed directly and works in bad weather; the camera recognises what the object is. Our numbers show it: radar added +0.056 NDS and cut velocity error by 0.33 m/s; the camera added +0.039 NDS.

Q: What exactly is feature selection, in one sentence? A: A small network that, before fusion, multiplies every LiDAR and radar feature by two learned numbers between 0 and 1: one for *which feature* (channel gate) and one for *where* (spatial gate).

Q: How is that different from attention fusion? A: Attention fusion shares trust between sensors per cell (weights sum to 1). Selection acts per sensor on channels and cells before fusion, has explicit sparsity, and can switch a feature off entirely.

Q: Why is it "camera-guided"? A: The camera BEV is an extra input to the gate networks; it carries semantic context ("this looks like a car") that helps decide what LiDAR and radar should keep. The camera itself is not gated.

Q: Why not use a Transformer for fusion? A: 200 × 200 cells = 40 000 tokens → 1.6 billion attention entries, which doesn't fit 4 GB. Gating is cheap (0.11 M parameters, ≈ 7 ms) and directly interpretable.

Q: What prevents the gates from simply closing everything? A: The coverage loss: at real object centres at least one sensor must stay open. The detection loss also suffers if useful information is removed.

About the results

Q: So your hypothesis failed? A: It is not supported *on nuScenes-mini*: E vs C is −0.061 NDS. With 323 training frames and one seed that is not a final verdict. The proper test is full nuScenes with three seeds, which the code already supports with one command.

Q: Then what is your contribution? A: (1) A complete, reproducible camera–LiDAR–radar pipeline from raw data to planning that runs on a 4 GB GPU; (2) an interpretable selection module whose gates show *which* sensor features matter *where*; (3) a controlled ablation (identical recipe, final-epoch evaluation, official devkit check) with an honest negative result and clear evidence that radar and camera help.

Q: Why are your scores so much lower than published papers? A: They train on 28 130 frames with large GPUs and big images; we use 323 frames, 128 × 352 images and 20 epochs on a 4 GB laptop GPU. The value here is the relative comparison under identical conditions.

Q: How do you know your evaluation is correct? A: Our metric code is a port of the nuScenes metrics, and model F was cross-checked with the official devkit: NDS 0.1028 vs 0.0943 (difference explained by the score threshold).

Q: Why did you evaluate the last epoch and not the best one? A: Choosing the best epoch on the evaluation set is tuning on the test data and adds luck. The last epoch gives every experiment the same budget. (The best-epoch protocol gives the same conclusions.)

Q: Why does model F have a higher collision rate (13 %) than E (7 %)? A: The collision differences between variants are small and inconsistent on 69 planning samples (one collision ≈ 1.4 %). The robust planning result is that all learned planners beat constant velocity.

About the engineering

Q: How do you handle sensors firing at different times? A: Each measurement goes sensor → car at its own timestamp → world → car at the LiDAR key-frame time. The world frame is fixed, so ego-motion is compensated.

Q: How does the camera know depth? A: It predicts a probability over 59 depth bins per image feature; during training LiDAR points projected into the image give the correct bin.

Q: Why fp32 and not mixed precision? A: On the GTX 1650 with cuDNN 9.10, fp16 3 × 3 convolutions with 256 input channels return NaN, and fp16 was 3.4× slower (no tensor cores). A start-up self-test checks this.

Q: Is it real-time? A: Model F takes ≈ 104 ms per frame (≈ 10 Hz) on a GTX 1650: camera 27 ms, LiDAR 18, radar 15, selection 7, fusion 19, backbone 11, detection 3, planning 4. nuScenes key-frames come at 2 Hz.

Q: How big is the model? A: 22.6 M parameters, 16.5 M of which are the camera branch (ResNet-18). Feature selection: 0.11 M.

Q: Is the planner tested in a simulator? A: No. It is *open-loop*: the plan is compared with the logged human path and real future object positions. Closed-loop testing would need a simulator (e.g. CARLA) and is future work.

Q: Did the model see the demo scenes during training? A: No. The demo uses scene-0103 and scene-0916 from the validation split; training scenes are flagged.

About the future

Q: What would you do next? A: Train on full nuScenes with three seeds per experiment on an RTX-class GPU; train E without the selection losses to separate the gating from the regulariser; try hard 0/1 gates; then closed-loop testing.

14. Limitations and Next Steps

Being open about limitations makes your answers stronger. These are the real ones:

Limitation	Consequence	Next step
nuScenes-mini (323 train / 81 val frames)	Low absolute scores; differences may be noise	Full nuScenes (28 130 train frames)
One seed per experiment	Seed-to-seed variance unknown	≥ 3 seeds per configuration
4 GB GPU: small images, few sweeps, batch 1	Weaker camera branch, slower training	RTX-class GPU (profiles already in `configs/hardware/`)
Sparsity target fixed at 35 % of cells	May remove information on small data	Run E with the selection losses switched off; tune the targets
Open-loop planning only	No feedback from the car's own actions	Closed-loop simulation
No map used by default	Planner does not know lanes or kerbs	Enable the map prior (`planning.occupancy.map_prior`)
Rare classes (bus, motorcycle, bicycle, cone) score AP 0	mAP comes almost entirely from car and pedestrian	More data, class-balanced sampling

Commands for the two most important next experiments:

```
python main.py train --stage joint --experiment E_lidar_radar_fs --set loss.lambda_feature_selection=0.0

python main.py ablation --train --epochs 20 --set dataset.version=v1.0-trainval dataset.train_split=train dataset.va
↪    l_split=val runtime.hardware_profile=rtx3060
```

15. One-Page Cheat Sheet

Item	Value
Dataset	nuScenes-mini: 10 scenes, 323 train / 81 val frames; val scenes scene-0103, scene-0916
Sensors	6 cameras · 1 LiDAR (32 beams) · 5 radars
BEV grid	±50 m, 0.5 m cells → 200 × 200; detection map 100 × 100 at 1 m
Camera branch	ResNet-18, images 128 × 352, 59 depth bins (1–60 m), LSS + depth supervision → (64, 200, 200)
LiDAR branch	5 sweeps, points (x, y, z, intensity, time-lag), dynamic PointPillars + 4 raster channels → 64 ch
Radar branch	≈ 400–800 points, 10 values each, pillars + dilations 1/2/4 + 5 raster channels (vx, vy …) → 64 ch
Feature selection	channel gate (64) + spatial gate (200 × 200) per sensor; F = F ⊙ c ⊙ s; keep targets 35 % cells / 75 % channels; gates start at 0.88
FS loss	coverage + 0.5 sparsity + 0.1 binarization, weight 0.5 in the total loss
Fusion	attention-gated (softmax trust per cell) → 128 ch; backbone SECOND-FPN → (256, 100, 100)
Detection	CenterPoint heatmap, 10 classes, box = 9 numbers (x, y, z, l, w, h, yaw, vx, vy)
Planning	6 waypoints × 0.5 s = 3 s; classical A* + iLQR, learned (attention + GRU residuals), hybrid
Model size	22.6 M parameters (camera 16.5 M, feature selection 0.11 M)
Speed / memory	≈ 104 ms per frame; training peak 3.2 GB, inference 0.77 GB (GTX 1650)
Training (ablation)	joint, 20 epochs, seed 42, AdamW $2 \cdot 10^{-4}$ cosine, batch 1 × accum 4, fp32, last epoch evaluated

Key results to remember

Comparison	Effect
Model F (official devkit)	NDS 0.1028 · mAP 0.0459
Best NDS / best mAP	C concat 0.116 / F 0.044
Feature selection vs concat (E − C)	−0.061 NDS → hypothesis not supported on mini
Radar added to LiDAR (C − A)	+0.056 NDS, −0.33 m/s velocity error
Camera added (F − E)	+0.039 NDS, +0.017 mAP
Learned planner vs constant velocity	ADE 1.26–1.53 m vs 1.77 m
What the gates keep (F)	34 % of LiDAR, 11 % of radar features; radar velocity channels kept (0.96–0.99)

Where things are in the code

Part	File
Whole model	`models/fusion_model.py`
Camera → BEV	`models/camera_bev/lss.py`
Feature selection + losses	`models/feature_selection/selection.py`, `losses.py`
Fusion methods	`models/fusion/fusion_methods.py`
Detection head	`models/detection/center_head.py`
Planners / tracker	`planning/trajectory_planner.py`, `planning/tracking.py`
Training	`training/trainer.py`, stages in `configs/training.yaml`
Experiments A–F	`configs/ablation.yaml`, results in `outputs/ablation/`
Dashboard	`app/streamlit_app.py`